Merge vs rebase

Objectives

- Describe how a merge joins diverged histories with a merge commit, while a fast-forward merge only moves the branch pointer
- Describe how a rebase replays your commits as new commits on a new base
- Apply the rule: do not rebase commits other people already have

Teaching

Start with a guess

While you worked on feature, main gained a commit. There are two ways to bring that work into your branch. The model shows the starting history.

Two ways to take in main

Both commands bring main's new work into your branch.

Merge: git merge main keeps every existing commit and adds a merge commit with two parents.

Rebase: git rebase main replays your commits on top of main's tip. Each replayed commit is new, with a new ID (C3 becomes C3′); the old ones are abandoned. History becomes a straight line.

The rule: Because rebase replaces commits, never rebase commits other people already have. Merging is always safe.

Apply it to a shared branch

Same two tools; now a teammate depends on your commits.

Model

Practice repositorySimulation: no real Git runs

1. C4Fix taxmain
2. C3Add searchHEAD → feature
3. C2Add cart

Recap

- git merge main keeps all commits and adds a merge commit with two parents.
- git rebase main replays your commits as new commits on main's tip, making a straight line.
- Rebased commits get new IDs; the old ones are abandoned.
- Never rebase commits others already have. When in doubt, merge.

Next topic: Pull requests and review comments

Cheat sheet

git merge main	join main into your branch with a merge commit
git rebase main	replay your commits on top of main
C3 vs C3′	same change, different commit after a rebase

Sources

Teaching claims were checked on 2026-10-06 (UTC) against live official documentation: the Git reference manual on git-scm.com (latest source release 2.56.0, released 2026-09-28). Simulated command output is illustrative: its exact wording was not checked and varies with Git version and settings.

- git-rebase: description
- git-rebase: recovering from upstream rebase
- git-merge: description

Static interaction alternatives

predict

Prediction (not scored). Your feature has C3; main gained C4 since you branched. On feature you run git rebase main. What happens to C3? Options: A new commit C3′ with the same change lands on top of C4; C3 itself moves, unchanged, on top of C4; C3 and C4 combine into one merge commit; C3 is deleted and its change lost. Answer: A new commit C3′ with the same change lands on top of C4. A new commit C3′ with the same change lands on top of C4: Rebase replays your change as a brand-new commit on main's tip. The old C3 is abandoned. C3 itself moves, unchanged, on top of C4: Attractive, but a commit's parent is part of its identity, so Git must create a new commit, C3′. C3 and

C4 combine into one merge commit: That describes a merge, not a rebase. C3 is deleted and its change lost: The change survives in C3′; only the old commit is left behind.

terminal

Practice terminal (simulated; nothing runs on your computer). Task: Try both from the same starting history and compare the results. Goals: Merge main into feature; find the merge commit; From the same start, rebase; find C3′. One sequence that meets every goal: git merge main, then Restore the starting history (practice only), then git rebase main.

mcq

Check (one retry, then the answer is shown). You pushed feature, and a teammate already built commits on top of it. You need main's new commits in feature. Which do you use? Options: git merge main; git rebase main; Either; both give the same history; git reset --hard main. Answer: git merge main. git merge main: Right: merge keeps every commit your teammate built on, adding one merge commit. git rebase main: Rebase replaces your pushed commits with new ones, so your teammate's work sits on commits that are no longer in your branch. Either; both give the same history: They differ: merge adds a merge commit; rebase creates new copies of your commits. git reset --hard main: That would throw away your feature commits entirely. Retry question: The model shows a history after someone took in main. Which command produced it? Options: git rebase main: C3 was replayed as C3′; git merge main: there is a merge commit; git merge main: C3 was kept as it was; Neither; nothing changed. Answer: git rebase main: C3 was replayed as C3′. git rebase main: C3 was replayed as C3′: Right: C3′ is a new commit on top of C4, and the old C3 is left with no branch. git merge main: there is a merge commit: No commit here has two parents, so no merge happened. git merge main: C3 was kept as it was: The branch now points at C3′, a different commit from C3. Neither; nothing changed: feature moved from C3 to C3′, so something did change.